

CONFIDE COMPUTER ACADEMY

Python Data Types

A Complete Guide with Examples

All 9 data types · 80+ code examples · Beginner to Intermediate

In Python, every value has a data type. A data type tells Python what kind of information a variable holds and what operations can be performed on it. Understanding data types is foundational — almost every program you write will depend on using the right type for the right job.

Python has two main categories of data types:

- Primitive types — hold a single simple value: str, int, float, bool, None
- Collection types — hold multiple values: list, tuple, dict, set

KEY CONC EPT

Python is dynamically typed — you never need to declare a type. Python figures it out from the value you assign. You can also check any value's type using the built-in `type()` function.

```
# Python figures out the type automatically
name = "Amaka"      # Python knows this is a str
age = 22             # Python knows this is an int
score = 92.5         # Python knows this is a float

# Check any type with type()
print(type(name))    # <class 'str'>
print(type(age))     # <class 'int'>
print(type(score))   # <class 'float'>
```

Quick Overview — All 9 Data Types

Type	Category	Example	Used For
<code>str</code>	Primitive	"Hello, Lagos"	Text, names, messages
<code>int</code>	Primitive	42	Whole numbers, counts, IDs
<code>float</code>	Primitive	3.14	Decimals, prices, measurements
<code>bool</code>	Primitive	True	Yes/No flags, conditions
<code>None</code>	Primitive	None	Absence of a value
<code>list</code>	Collection	[1, 2, 3]	Ordered changeable sequences
<code>tuple</code>	Collection	(1, 2, 3)	Ordered fixed sequences
<code>dict</code>	Collection	{'name': 'Amaka'}	Key-value pairs, records

<code>set</code>	Collection	{1, 2, 3}	Unique unordered values
------------------	------------	-----------	-------------------------

TYPE 1 String — Text Data

str**"A"**

Primitive Type

str

A string is any sequence of characters — letters, numbers, spaces, symbols. Strings are immutable: once created, you cannot change individual characters.

Creating Strings

Strings can be created with single quotes, double quotes, or triple quotes for multi-line text.

```
# Single or double quotes - no difference
first_name = "Amaka"
last_name = 'Okonkwo'

# Triple quotes - spans multiple lines
address = """
14 Ero Crescent
Gloryland Estate
Lagos"""

# Empty string
empty = ""

# String with special characters
line1 = "First line\nSecond line"    # \n = new line
tab = "Name:\tAmaka"                 # \t = tab
quote = "She said \"Hello\""         # escaped quote
```

f-strings — The Modern Way to Format

f-strings (formatted string literals) let you embed Python expressions directly inside a string. They are the preferred way to build strings with variable values.

```
name = 'Amaka'
age = 22
score = 92.5

# Basic f-string - prefix the string with f
print(f"Hello, {name}!")    # Hello, Amaka!
print(f"Age: {age}")        # Age: 22

# Expressions inside f-strings
print(f"In 5 years: {age + 5}")    # In 5 years: 27
print(f"Score: {score:.1f}%")    # Score: 92.5%
print(f"Name: {name.upper()}")    # Name: AMAKA

# Padding and alignment
print(f"{name!s:<10} {score:>6.1f}") # aligned output
```

Output:

```
Hello, Amaka!
Age: 22
In 5 years: 27
Score: 92.5%
Name: AMAKA
```

Essential String Methods

String methods do not modify the original string — they always return a new one.

```
text = " Hello, Lagos! "
```

Case

```
print(text.upper())      # ' HELLO, LAGOS! '
print(text.lower())      # ' hello, lagos! '
print(text.title())      # ' Hello, Lagos! '
```

Whitespace

```
print(text.strip())      # 'Hello, Lagos!' - removes spaces both ends
print(text.lstrip())     # 'Hello, Lagos! ' - left only
print(text.rstrip())     # ' Hello, Lagos!' - right only
```

Search

```
print('Lagos' in text)   # True
print(text.find('Lagos')) # 9 (index position)
print(text.count('l'))   # 2
print(text.startswith(' Hello')) # True
```

Modify

```
print(text.replace('Lagos', 'Abuja')) # ' Hello, Abuja! '
print(text.strip().split(', '))       # ['Hello', 'Lagos!']
```

Split and join

```
words = 'apple,banana,orange'.split(',') # ['apple','banana','orange']
joined = ' | '.join(words)               # 'apple | banana | orange'
```

String Indexing & Slicing

Every character in a string has a position number called an index, starting from 0.

```
name = "Confide"
#   C o n f i d e
#   0 1 2 3 4 5 6  ← positive index
#  -7-6-5-4-3-2-1  ← negative index

print(name[0])      # C - first character
print(name[3])      # f - fourth character
print(name[-1])     # e - last character
print(name[-3])     # i - third from last
```

Slicing — name[start:stop:step]

```
print(name[0:3])    # Con - chars 0, 1, 2 (stop not included)
print(name[3:])     # fide - from index 3 to end
print(name[:4])     # Conf - from start to index 3
print(name[::2])    # Cni - every 2nd character
print(name[::-1])   # edifnoC - reversed!
```

TYPE 2 Integer — Whole Numbers

int

42

Primitive Type

int

An integer is any whole number — positive, negative, or zero. Python integers have no size limit: they can be as large as your computer's memory allows.

Creating Integers

```
# Basic integers
students = 30
temperature = -5
zero = 0

# Underscores for readability (Python 3.6+)
population = 200_000_000 # same as 200000000
salary = 150_000

# Different number bases
binary = 0b1010 # binary → 10
octal = 0o17 # octal → 15
hex_val = 0xFF # hex → 255

print(binary, octal, hex_val) # 10 15 255
```

Integer Arithmetic

```
a = 17
b = 5

print(a + b) # 22 - addition
print(a - b) # 12 - subtraction
print(a * b) # 85 - multiplication
print(a / b) # 3.4 - division (ALWAYS returns float!)
print(a // b) # 3 - floor division (integer result)
print(a % b) # 2 - modulus (remainder)
print(a ** b) # 1419857 - exponentiation (17 to the power 5)

# Practical use of modulus — check even or odd
number = 42
if number % 2 == 0:
    print(f"{number} is even")
else:
    print(f"{number} is odd")

# Increment patterns
count = 0
count += 1 # count = count + 1 → 1
count += 1 # → 2
count *= 3 # count = count * 3 → 6
```

Output:

```
22
12
85
3.4
3
```

```
2
1419857
42 is even
```

Converting To and From int

```
# Convert string to int
age_str = input("Enter your age: ") # input() ALWAYS returns str
age = int(age_str)                  # convert to int to do maths
print(age + 1)                      # next year

# Convert float to int - truncates (does NOT round)
print(int(3.9))    # 3 - not 4!
print(int(-3.9))   # -3 - not -4!

# Convert bool to int
print(int(True))   # 1
print(int(False))  # 0

# Convert int to string
score = 92
message = "Score: " + str(score)    # "Score: 92"

# Useful int functions
print(abs(-45))    # 45 - absolute value
print(pow(2, 8))    # 256 - same as 2**8
print(max(3, 7, 1, 9)) # 9
print(min(3, 7, 1, 9)) # 1
```

TYPE 3 Float — Decimal Numbers

float

3.14

Primitive Type

float

A float (floating-point number) holds decimal values. Whenever you divide two integers in Python, the result is always a float — even if it divides evenly.

Creating Floats

```
# Basic floats
price      = 4999.99
pi         = 3.14159
temperature = -0.5
salary     = 150000.0    # .0 makes it explicitly a float

# Scientific notation
tiny       = 1.5e-4      # 0.00015
huge       = 2.5e8       # 250000000.0

# Division always returns float
print(10 / 2)           # 5.0 ← float, not 5
print(10 / 3)           # 3.3333333333333335

# Float arithmetic
print(4999.99 * 1.075)   # with 7.5% tax
print(round(4999.99 * 1.075, 2)) # rounded to 2 decimal places
```

Rounding and Precision

IMPORTANT

Floats can have tiny precision errors due to how computers store binary fractions. For example, $0.1 + 0.2$ is not exactly 0.3 . Use `round()` for display, or the `decimal` module for financial calculations.

```
# round(number, decimal_places)
print(round(3.14159, 2))    # 3.14
print(round(3.14159, 4))    # 3.1416
print(round(7.5))           # 8 (rounds to nearest even)
print(round(2.5))           # 2 (rounds to nearest even)

# The famous float precision issue
print(0.1 + 0.2)            # 0.30000000000000004 ← not exactly 0.3!
print(round(0.1 + 0.2, 2))  # 0.3 ← use round() for display

# Formatting floats in output
price = 4999.99
print(f"Price: {price:.2f}") # Price: 4999.99 (2 decimal places)
print(f"Price: {price:,.2f}") # Price: 4,999.99 (with comma)
print(f"Price: N{price:,.0f}") # Price: N4,999,999

import math
print(math.floor(4.9))       # 4 — rounds DOWN always
print(math.ceil(4.1))        # 5 — rounds UP always
print(math.sqrt(144))        # 12.0 — square root
```

TYPE 4 Boolean — True or False

bool**T/F**

Primitive Type

bool

A boolean holds exactly one of two values: True or False. It is the foundation of all decision-making in Python — every if statement evaluates to a boolean.

Boolean Values and Logic

```
# The two boolean values
is_enrolled = True
has_paid    = False

# Booleans come from comparisons
print(10 > 5)      # True
print(10 == 5)     # False
print(10 != 5)     # True
print(10 >= 10)    # True

# Logical operators: and, or, not
age = 20
score = 75

print(age >= 18 and score >= 60) # True - BOTH must be True
print(age < 18 or score >= 60)  # True - at least ONE must be True
print(not True)                 # False - flips the value
print(not False)                # True

# Using booleans in if statements
is_raining = True
if is_raining:
    print("Take an umbrella!")
if not is_raining:
    print("Leave the umbrella at home.")
```

Truthy and Falsy Values

In Python, every value can be evaluated as either truthy or falsy in a boolean context. You don't have to write `== True` explicitly — Python evaluates the value itself.

RULE

These values are always FALSY: 0, 0.0, "" (empty string), [] (empty list), {} (empty dict), () (empty tuple), None. Everything else is TRUTHY.

```
# Falsy values - these all act like False
if 0:      print('never prints') # 0 is falsy
if '':     print('never prints') # empty string is falsy
if []:     print('never prints') # empty list is falsy
if None:   print('never prints') # None is falsy

# Truthy values - these all act like True
if 42:     print('prints!')      # any non-zero number
if 'hello': print('prints!')     # non-empty string
if [1,2,3]: print('prints!')     # non-empty list

# Practical use - check if list has items
students = ['Amaka', 'Tunde']
if students: # truthy because not empty
```



```
print(f"We have {len(students)} students")

empty_list = []
if not empty_list:           # falsy, so 'not falsy' = True
    print("No students enrolled yet")

# bool() — explicitly convert to boolean
print(bool(0))               # False
print(bool(1))               # True
print(bool(''))              # False
print(bool('hi'))            # True
print(bool([]))              # False
```

TYPE 5 NoneType — The Absence of a Value

None



Primitive Type

None

None represents the intentional absence of any value. It is not the same as 0, False, or an empty string — it means 'nothing is here'. It is Python's equivalent of null in other languages.

Using None

```
# None is its own type
result = None
print(type(result))    # <class 'NoneType'>

# Functions return None by default if no return statement
def greet(name):
    print(f"Hello, {name}!")    # no return statement

value = greet('Amaka')    # prints: Hello, Amaka!
print(value)              # None — nothing was returned

# Checking for None — ALWAYS use 'is' not '=='
data = None
if data is None:
    print("No data received yet")

if data is not None:
    print(f"Data: {data}")

# Common use — placeholder before a value is known
def find_student(email, students):
    found = None    # start with no result
    for s in students:
        if s['email'] == email:
            found = s    # update when found
    return found    # None if not found

result = find_student('x@x.com', [])
if result is None:
    print("Student not found")
```

TYPE 6 List — Ordered, Changeable Sequence

list

[]Collection
Type**list**

A list stores multiple items in a specific order. Items can be added, removed, or changed after creation. Lists can hold any mix of types — including other lists.

Creating and Accessing Lists

```
# Creating lists
fruits = ['apple', 'banana', 'orange', 'mango']
scores = [92, 78, 88, 65, 71]
mixed = ['Amaka', 22, True, 3.14, None] # any types
nested = [[1,2,3], [4,5,6], [7,8,9]] # list of lists
empty = []

# Access by index (starts at 0)
print(fruits[0]) # apple - first
print(fruits[-1]) # mango - last
print(fruits[-2]) # orange - second to last

# Slicing - [start:stop:step]
print(fruits[1:3]) # ['banana', 'orange']
print(fruits[:2]) # ['apple', 'banana']
print(fruits[2:]) # ['orange', 'mango']
print(fruits[::-1]) # ['mango', 'orange', 'banana', 'apple'] reversed

# Access nested list
print(nested[1][2]) # 6 - row 1, column 2
```

Modifying Lists

```
students = ['Amaka', 'Tunde', 'Ngozi']

# Add items
students.append('Emeka') # add to END
students.insert(1, 'Bola') # insert at index 1
students.extend(['Kemi', 'Seun']) # add multiple to end

# Remove items
students.remove('Tunde') # remove by VALUE
removed = students.pop() # remove and return LAST item
removed = students.pop(0) # remove and return item at index 0
del students[1] # delete by index
students.clear() # remove ALL items

# Change items
fruits = ['apple', 'banana', 'orange']
fruits[1] = 'grape' # change single item
fruits[0:2] = ['kiwi', 'mango'] # change slice

# Sorting
numbers = [3, 1, 4, 1, 5, 9, 2]
numbers.sort() # sort in place: [1,1,2,3,4,5,9]
numbers.sort(reverse=True) # descending: [9,5,4,3,2,1,1]
sorted_copy = sorted(numbers) # returns NEW sorted list
numbers.reverse() # reverse in place
```

Essential List Methods

```
numbers = [3, 1, 4, 1, 5, 9, 2, 6]

print(len(numbers))          # 8 – number of items
print(sum(numbers))          # 31 – add all numbers
print(max(numbers))          # 9 – largest
print(min(numbers))          # 1 – smallest
print(numbers.count(1))      # 2 – how many times 1 appears
print(numbers.index(5))      # 4 – index of first 5
print(4 in numbers)          # True – membership check
print(7 in numbers)          # False

# List comprehension – concise way to build lists
squares = [x**2 for x in range(1, 6)]
# [1, 4, 9, 16, 25]

evens = [x for x in range(1, 11) if x % 2 == 0]
# [2, 4, 6, 8, 10]

names = ['amaka', 'tunde', 'ngozi']
upper_names = [n.title() for n in names]
# ['Amaka', 'Tunde', 'Ngozi']
```

TYPE 7 Tuple — Ordered, Fixed Sequence

tuple

()

Collection
Type

tuple

A tuple is like a list, but immutable — once created, it cannot be changed. Use tuples for data that should never be modified: coordinates, RGB colours, database records.

Creating and Using Tuples

```
# Creating tuples - parentheses (optional but recommended)
point = (10, 20) # x, y coordinate
rgb_red = (255, 0, 0) # colour
student = ('Amaka', 22, 'A', 'Lagos') # fixed record
single = (42,) # MUST have trailing comma for single item!
also_tuple = 1, 2, 3 # parentheses optional
empty = ()

# Access - same as list
print(point[0]) # 10
print(point[-1]) # 20
print(student[1:3]) # (22, 'A')

# Tuples are IMMUTABLE - this causes an error:
# point[0] = 99 # TypeError: 'tuple' object does not support item assignment

# Tuple unpacking - very useful!
x, y = point
print(f'x={x}, y={y}') # x=10, y=20

name, age, grade, city = student
print(f'{name} is {age} years old') # Amaka is 22 years old

# Swap variables using tuple unpacking
a, b = 1, 2
a, b = b, a # swap!
print(a, b) # 2 1
```

When to Use tuple vs list

	list []	tuple ()
Mutable (changeable)?	Yes — items can be added/removed/changed	No — contents fixed after creation
Faster?	Slightly slower	Slightly faster (Python optimises tuples)
Can be dict key?	No — lists are not hashable	Yes — tuples can be dict keys
Best used for...	Shopping cart, class roll, todo list	Coordinates, colours, database rows

TYPE 8 Dictionary — Key-Value Pairs

dict

{ }

Collection
Type**dict**

A dictionary stores data as key-value pairs. You look up values by their key — not by position. Keys must be unique. Dicts are the most important collection type in Python web development.

Creating and Accessing Dictionaries

```
# Creating a dictionary
student = {
    'name' : 'Amaka Okonkwo',
    'age' : 22,
    'grade' : 'A',
    'grades': [92, 78, 88, 65],
    'active': True
}

# Access by key
print(student['name'])    # Amaka Okonkwo
print(student['age'])     # 22

# Safe access with .get() — returns None if key doesn't exist
print(student.get('email'))    # None — no KeyError!
print(student.get('email', 'N/A')) # N/A — custom default

# Check if key exists
print('name' in student)    # True
print('email' in student)  # False
```

Modifying Dictionaries

```
student = {'name': 'Amaka', 'age': 22}

# Add new key
student['city'] = 'Lagos'
student['email'] = 'amaka@email.com'

# Update existing key
student['age'] = 23

# Remove keys
del student['email']    # delete key (error if missing)
city = student.pop('city') # remove + return value
city = student.pop('city', 'Unknown') # safe — default if missing
student.clear()         # remove ALL keys

# Update multiple keys at once
student.update({'age': 24, 'grade': 'A+', 'city': 'Abuja'})

# Merge two dicts (Python 3.9+)
extra = {'course': 'Backend Dev', 'year': 2025}
combined = student | extra    # creates new merged dict
```

Iterating Dictionaries

```
student = {'name': 'Amaka', 'age': 22, 'city': 'Lagos'}

# Iterate keys (default)
for key in student:
    print(key)           # name  age  city

# Iterate values
for value in student.values():
    print(value)         # Amaka  22  Lagos

# Iterate key-value pairs — most common
for key, value in student.items():
    print(f"{key}: {value}")
# name: Amaka
# age: 22
# city: Lagos

# List of all keys and values
keys = list(student.keys()) # ['name', 'age', 'city']
values = list(student.values()) # ['Amaka', 22, 'Lagos']

# Dict comprehension
scores = {'Amaka': 92, 'Tunde': 78, 'Ngozi': 88}
passed = {name: score for name, score in scores.items() if score >= 80}
# {'Amaka': 92, 'Ngozi': 88}
```

TYPE 9 Set — Unique Unordered Values

set

{ }

Collection
Type**set**

A set stores unique values with no particular order and no duplicates. Sets are extremely fast at checking membership (in), and support powerful mathematical set operations.

Creating and Using Sets

```
# Creating sets
fruits = {'apple', 'banana', 'orange', 'apple'} # duplicate removed!
print(fruits) # {'apple', 'banana', 'orange'} — order NOT guaranteed

# Create set from list — removes duplicates
grades = [85, 92, 78, 85, 65, 92, 88]
unique = set(grades)
print(unique) # {65, 78, 85, 88, 92} — each appears once

# Empty set — MUST use set(), NOT {} (that's a dict!)
empty = set()

# Membership — sets are very fast at this
cities = {'Lagos', 'Abuja', 'Kano', 'Port Harcourt'}
print('Lagos' in cities) # True
print('Ibadan' in cities) # False

# Add and remove
cities.add('Ibadan') # add one item
cities.update(['Enugu', 'Ondo']) # add multiple
cities.remove('Kano') # remove — error if not found
cities.discard('Kano') # remove — NO error if not found
```

Set Operations — The Superpower

Sets let you combine and compare collections using mathematical operations.

```
python_students = {'Amaka', 'Tunde', 'Ngozi', 'Emeka'}
flask_students = {'Ngozi', 'Emeka', 'Bola', 'Kemi'}

# UNION — all students in either group
all_students = python_students | flask_students
# or: python_students.union(flask_students)
print(all_students)
# {'Amaka', 'Tunde', 'Ngozi', 'Emeka', 'Bola', 'Kemi'}

# INTERSECTION — students in BOTH groups
both = python_students & flask_students
# or: python_students.intersection(flask_students)
print(both) # {'Ngozi', 'Emeka'}

# DIFFERENCE — in python_students but NOT flask_students
only_python = python_students - flask_students
# or: python_students.difference(flask_students)
print(only_python) # {'Amaka', 'Tunde'}

# SYMMETRIC DIFFERENCE — in one group but NOT both
exclusive = python_students ^ flask_students
print(exclusive) # {'Amaka', 'Tunde', 'Bola', 'Kemi'}
```



```
# Subset and superset
a = {1, 2}
b = {1, 2, 3, 4}
print(a.issubset(b))    # True — all of a is in b
print(b.issuperset(a))  # True — b contains all of a
```

BONUS CHAPTER · ESSENTIAL SKILL

Type Conversion

Converting between types is one of the most common tasks in Python programming. This is especially important because `input()` always returns a string — you must convert it before doing maths.

Explicit Conversion (Casting)

```
# String → other types
int('42')          # 42
float('3.14')      # 3.14
bool('hello')      # True (any non-empty string)
bool('')           # False (empty string)
list('hello')       # ['h', 'e', 'l', 'l', 'o']

# Number → string
str(42)            # '42'
str(3.14)          # '3.14'
str(True)          # 'True'

# Number → int / float
int(3.9)           # 3 (truncates, does NOT round)
int(True)          # 1
int(False)         # 0
float(42)           # 42.0
float(True)        # 1.0

# List / tuple / set conversions
my_list = [1, 2, 2, 3, 3]
my_set = set(my_list) # {1, 2, 3} - removes duplicates
my_tuple = tuple(my_list) # (1, 2, 2, 3, 3)
back_to_list = list(my_set) # [1, 2, 3] - order not guaranteed

# Practical example - user input
age_input = input("Enter your age: ") # always returns str!
age = int(age_input) # convert to int
next_year = age + 1 # now we can do maths
print(f"Next year you will be {next_year}")
```

Output:

```
int('42')      → 42
float('3.14')  → 3.14
int(3.9)       → 3
str(42)        → '42'
set([1,2,2,3]) → {1, 2, 3}
```

Common Conversion Errors and How to Avoid Them

```
# These will raise errors - always validate first!

# int('hello')    → ValueError: invalid literal for int()
# int('3.14')     → ValueError: int() can't convert float string
# float('hello')  → ValueError

# Safe conversion - wrap in try/except
def safe_int(value):
    """Convert to int safely, return None if impossible."""
    try:
        return int(value)
```

```
    except (ValueError, TypeError):
        return None

print(safe_int("42"))      # 42
print(safe_int("hello"))  # None — no crash!
print(safe_int(3.9))      # 3

# Always check before converting user input
user_input = input("Enter a number: ")
if user_input.isdigit():
    number = int(user_input)
    print(f"Doubled: {number * 2}")
else:
    print("That is not a whole number!")
```

QUICK REFERENCE

All 9 Data Types at a Glance

```

# --- PRIMITIVE TYPES ---
str      = "Hello" 'World' f'{var}' str.upper() .split() .strip()
int      = 42 -7 0 1_000_000 a+b a-b a*b a//b a%b a**b
float    = 3.14 -0.5 2e8 round(x,2) math.floor() math.ceil()
bool     = True False and or not == != < > <= >=
None     = None - use: x is None / x is not None

# --- COLLECTION TYPES ---
list     = [1,'a',True] mutable ordered
          .append() .insert() .remove() .pop() .sort() .reverse()
          [start:stop:step] len() sum() max() min()
          [x**2 for x in lst if x > 0] # comprehension

tuple    = (1,'a',True) immutable ordered
          a,b = (1,2) # unpacking
          a,b = b,a # swap

dict     = {'key': 'value'} key-value pairs
          d['key'] d.get('key','default') 'key' in d
          d.keys() d.values() d.items() d.update() d.pop()
          {k:v for k,v in d.items() if condition} # comprehension

set      = {1,2,3} unique unordered
          .add() .discard() .update()
          a | b a & b a - b a ^ b (union/intersect/diff/sym)

# --- TYPE CONVERSION ---
int('42') float('3.14') str(42) bool(x) list(x) set(x) tuple(x)

# --- CHECK TYPE ---
type(x) # <class 'str'>
isinstance(x, str) # True/False
isinstance(x, (int,float)) # check multiple types

```

Confide Computer Academy

Python Data Types — A Complete Guide with Examples

© COPYRIGHT 2026 CONFIDE COMPUTER ACADEMY